

An Approach for Effort Estimation in Incremental Software Development using Cosmic Function Points

Freddy Paz

Pontificia Universidad Católica del
Perú

Av. Universitaria 1801, Lima 32, Perú
(511) 626-2000 - 4812
fpaz@pucp.pe

Claudia Zapata

Pontificia Universidad Católica del
Perú

Av. Universitaria 1801, Lima 32, Perú
(511) 626-2000 - 4819
zapata.cmp@pucp.pe

José Antonio Pow-Sang

Pontificia Universidad Católica del
Perú

Av. Universitaria 1801, Lima 32, Perú
(511) 626-2000 - 5111
japowsang@pucp.pe

ABSTRACT

Context: The current trend for software construction consists in developing software through increments, which is recommended by recognized methodologies such as Rational Unified Process or Agile Methodologies. Although Cosmic Functional Size Measurement Method is one of the most known techniques to estimate effort and cost in software projects, there is little evidence that indicates how to adapt it to non-waterfall lifecycles, such as incremental ones. *Goal:* This paper presents an approach called Incremental CFP that allows estimating effort for each increment using Cosmic Function Points (CFP) and COCOMO's effort adjustment factor (EAF). *Method:* In order to validate this new approach, we employed data from two different projects in which undergraduate students had to work originally with IFPUG function points. *Results:* The results were considered as promising given that the difference between the estimated effort and the real effort was lower than 30% for the second increment. *Conclusions:* With a minimum difference between the estimated effort and the real effort, it was possible to establish that the proposed technique is acceptable and predictive. However, more study cases should be performed before generalizing its applicability for different contexts.

Categories and Subject Descriptors

D.2.9 [Software]: Management—Time estimation

General Terms

Management, Measurement, Economics, Experimentation.

Keywords

Cosmic Function Points, Incremental Software Development, Object Oriented, COCOMO.

1. INTRODUCTION

Incremental software development is one of the most used and widely recognized software development models in recent times. The mechanism proposed by the incremental software development model mainly consists in dividing the complete

software development into small increments, so that they can be implemented in relatively small amounts of time. This approach is becoming more dominant in the informatics field because user requirements can be satisfied with frequent software deliverables, allowing these users to experience the increasing benefits of their requested solution rather quickly and constantly [8].

On the other hand, accurate software effort estimation is becoming extremely important to ensure the successful completion of any software development project. Effort estimation allows project managers to calculate costs and time in a precise way, which have a strong impact on software functional quality as a whole. However, estimation process still represents a challenge when dealing with waterfall development models.

Alternatively, Cosmic Functional Size Measurement Method [12] is a widely used and relatively new technique, which was developed by Common Software Measurement International Consortium in order to improve the measurement of functional size for a wide range of software. Similarly, COCOMO II, proposed and developed by Barry Boehm [2], is one of the best-documented and utilized models for cost estimation. The model proposes a set of cost drivers that indicates the context in which the project is currently set. These drivers are used to determine the Effort Adjustment Factor (EAF), which is utilized to adjust the effort required to develop a software system.

This paper presents an approach called Incremental CFP that allows the estimation of effort for each software increment using Cosmic Function Points (CFP) and COCOMO's EAF. This approach could be complemented with the Use Case Precedence Diagram to determine the sequence of construction use cases (or use case prioritization) [11].

The rest of the paper is organized as follows: Section 2 describes the related work in the area, Section 3 details the characteristics and procedures of Incremental CFP, Section 4 describes the scenarios for the studies performed in projects with undergraduate students, and Section 5 shows the obtained results. Finally, a summary and our plans for future research will conclude our paper.

2. RELATED WORK

We conducted a systematic review to find similar techniques to Incremental CFP. We could find three techniques that define procedures to estimate the effort in incremental lifecycles or similar lifecycles, such as the iterative. The techniques are proposed by Orr et al. [9], Kang et al. [6] and Hericko et al. [5].

Only two of the techniques [6] [5] define rules to estimate effort between increments or iterations. However, they correspond to an

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ESEM'14, September 18–19, 2014, Torino, Italy.

Copyright 2014 ACM 978-1-4503-2774-9/14/09...\$15.00.

<http://dx.doi.org/10.1145/2652524.2652571>

adaptation of functional size measurement method proposed by IFPUG and none of the techniques defines how to divide the number of function points between iterations or increments. Besides, none of them considered that the productivity of the team might change due to several factors such as staff turnover, software development experience, knowledge of the tools and languages, throughout the entire project. None of the techniques considered how to prioritize the construction of use cases or requirements. Additionally, it was observed that the technique proposed by Hericko et al. indicates how to calculate Function Points using object-oriented models.

3. THE TECHNIQUE

Incremental CFP is being proposed as a technique to determine the required effort to construct each software increment Cosmic Function Points (CFP). This technique is an adaptation of Incremental FP that utilizes IFPUG Function Points [8]. To achieve this goal, Incremental CFP provides guidelines to determine the Cosmic Function Points (CFP) for each increment. The criterion for adaptation of the Cosmic Functional Size Measurement technique to incremental life cycle models mainly consists in considering that the result of the CFP calculation for the whole project (a.k.a. Total_CFP) must be equal to the sum of CFPs calculated separately for each increment ($CFP(i) = CFP$ of increment "i"). This formula is shown in Formula 1.

$$Total_CFP = \sum_{i=1}^n CFP(i) \quad (1)$$

The next task is to define the software increments that will be developed along the entire project, and the effort required to construct each increment. We can use the Use Case Precedence Diagram [11] to select which use cases could be developed in each increment. In order to calculate the estimated effort for the first increment, we must calculate the estimated productivity. This productivity is calculated using historical productivity of previous projects, which is measured in men-hours (m-h)/CFP, and historical COCOMO's EAF. Formula 2 shows how to calculate the estimated productivity. Here, $EstProd_Incr1$ represents the estimated productivity for the first increment measured in m-h/CFP, EAF_Incr1 represents the EAF value calculated for the first increment, $HistoricProd$ represents the historical productivity value of a previous project, and $EAF_Historic$ represents the EAF value related to the historical productivity of the previous project.

$$EstProd_Incr1 = \frac{EAF_Incr1 \times HistoricProd}{EAF_Historic} \quad (2)$$

Formula 3 was adapted from Formula 2 in order to calculate the productivity value for next increments. Here, $EstProd_Incr(i)$ represents the estimated productivity for increment "i", $EAF(i)$ represents the EAF value calculated for increment "i" or increment "i-1", $RealProd_Incr(i-1)$ represents the real productivity value in the increment "i-1", and i is the number of increment.

$$EstProd_Incr(i) = \frac{EAF(i) \times RealProd_Incr(i-1)}{EAF(i-1)} \quad (3)$$

In Formula 3, the estimated productivity is calculated using the information obtained in the previous increment. However, it can

also use the average of real productivities obtained in previous increments, as it is presented in Formula 4 (j = number of the previous increment before increment i).

$$EstProd_Incr(i) = EAF(i) \times \frac{\sum_{j=1}^{i-1} RealProd_Incr(j)}{i-1} \quad (4)$$

The next task is to determine the number of CFPs for each increment using Formula 5. Here, $CFP_Incr(i)$ represents the number of CFPs for increment "i", $CFP_UC(k)$ represents the number of CFPs for use case "k" and k is the use case that will be developed in the increment "i".

$$CFP_Incr(i) = \sum_{k=1}^n CFP_UC(k) \quad (5)$$

Finally, the effort required to develop the next increment is calculated using Formula 6. Here, $EstEffort_Incr(i)$ represents the estimated effort for increment "i", $CFP_Incr(i)$ represents the number of CFPs for increment "i" obtained with Formula 5, and $EstProd_Incr(i)$ represents the estimated productivity for "i" obtained with Formula 3 or Formula 4.

$$EstEffort_Incr(i) = CFP_Incr(i) \times EstProd_Incr(i) \quad (6)$$

It could happen that the estimated effort for one increment could be greater or lesser than the development capacity of the team. In such cases, the team may change the number of use cases to be developed in each increment accordingly.

4. TECHNIQUE VALIDATION

We used the information collected from two projects developed by undergraduate students who had originally used IFPUG function points to estimate the effort required in their own projects [8]. The projects, whose results are shown, were developed by undergraduate students of the Informatics Engineering Program that were enrolled in a Software Engineering course.

4.1 Students

The students were divided randomly into two project teams consisting of 12 students. Each team had to have an equal proportion of male and female students and the average academic performance of the members in a team had to be similar among all teams. These criteria were considered since it was empirically proved that ignoring these issues in a team could lead to bias in project development performance. Additionally, it was ensured that all participants had the same knowledge and experience in software development because they all had to fulfill the same set of course requirements before entering the Software Engineering course.

4.2 Projects

The topics for the two projects were different in each group to avoid plagiarism. However, these projects were related to the implementation of a same kind of software: a transaction processing system [7]. In these systems, the input/output processes are similar. Therefore, results obtained from these two groups could be considered comparable. The topics presented for each one were: (1) an information system for a company that sells

tickets for any kind of event, and (2) an information system for library management in a higher education center. All data was collected from a previous study performed by Pow-Sang [10].

4.3 Project methodology

The methodology used by every team was an adaptation of RUP. Before beginning of the construction phase, each team had to complete the Software Requirements Specifications document (SRS) with use cases. To achieve this, a teaching assistant was assigned for each team, who was in charge of tutoring the students during the development of the software. All teaching assistants had been working on real software projects for at least 7 years. As a design consideration, client/server 2-tiered architecture was used and the whole project was validated through a prototype.

All teams had to perform three increments during the construction phase. The duration of each increment was approximately two weeks. For all the projects, students used Microsoft SQL Server as the preferred database manager, Microsoft Windows as the preferred development platform, and Microsoft Office for documentation purposes. The students used the Use Case Precedence Diagram (UCPD) [11] to determine the sequences of construction of the use cases, and the rules of Incremental CFP to determine Cosmic Function Points.

5. RESULTS

All projects were completed in a satisfactory way. There was no need to discard data considered unreliable. Table 1 shows the information obtained for each team. For each increment, the first column shows the real effort of each team in men-hours (m-h), and the second column shows the number of CFPs calculated (We used the SRS of each project in order to calculate CFP).

Table 1. Real Effort and CFP per Increment for Each Team

Team	Increment 1		Increment 2	
	Real Effort (m-h)	CFP	Real Effort (m-h)	CFP
A	248	213	238	272
B	313	111	170	122

In a detailed form, students had to fill in a report of the hours needed to work on a project task per day. The students were informed that the final grade was not influenced by the number of hours invested in the project. This was specified to ensure the honesty in the effort recorded by each team member.

Real effort values only consider effective work employed in software construction. To calculate the results shown in table 2, we used the magnitude of relative error (MRE) defined in Formula 7. Here, y is the value for real effort, and $\hat{y}$ is the value for estimated effort [3].

$$MRE = \frac{|y - \hat{y}|}{y} \quad (7)$$

According to Hastings et al. [4], the MRE value has to be analyzed using three criteria. (1) Less than 0.2, which indicates that the technique can be considered predictive and confidently used for software planning purposes. (2) Between 0.2 and 0.5, which indicates that it is acceptable, but must be used with caution. (3) More than 0.5, which indicates that is not trustworthy for software planning purposes.

Regarding the data obtained for each increment, we only have considered for analysis, the effort estimation data for the second increment. Estimation data for the first increment was not considered precise and trustworthy because data from previous semesters (i.e. not part of the teams' own data) were used in its calculation.

5.1 Second Increment Analysis

For the effort estimation process for the second increment, it was assumed that the changes obtained from context modification were mainly defined by the increase in knowledge of the programming environment and experience in software application development. Team members had worked in small projects in previous semesters developing small applications. The students had not used the programming tool before the construction of the first increment. At the beginning of the second increment, the students knew how to program with the tool (Visual Basic .Net). The change in the context between the first and the second increment can be represented with the COCOMO's effort multipliers LEXP (language experience) and AEXP (application experience). Both multipliers were considered "very low" in the first increment and "very high" in the second increment. Therefore, EAF of the first increment was 1.46 and EAF of the second increment was 0.8.

Table 2 shows the results obtained by applying Formula 2 to calculate the estimated productivity (first column) and estimated effort (second column). The last column shows the MRE.

Table 2. Productivity, Effort Estimated, and MRE for the Second Increment

Team	Estimated Productivity (m-h/CFP)	Estimated Effort (m-h)	Real Effort (m-h)	MRE
A	0.875	302	238	27%
B	0.800	189	170	10%

Regarding the differences between estimated and real effort measured with the MRE value, it can be observed in Table 2 that they are all below 30%. This means that, according Hastings et al. [4], Incremental CFP can be considered as an acceptable technique. Table 2 also shows that estimated productivities are different among teams. This means it is not advisable to use the data recorded from another team in order to estimate a team's effort. In any case, it is better for a team to use its own data.

5.2 Comparison between Incremental CFP and Global Estimations

According to Abrahamsson et al. [1], global estimation models use a list of variables (e.g. software size, design metrics, technological factors) to estimate the effort for a complete project derived from historical data. These models are usually employed to predict the development effort for projects that are similar in conception. However, the main characteristic of incremental estimation models is that they are redeployed on each increment, allowing for the accommodation of any change that could affect the effort estimation in a more convenient way.

In this context, we performed a comparison between a generic global estimation model and Incremental CFP, taking into consideration three criteria. (1) We only used the data obtained from the first increment. (2) The value of the estimated effort for

the first increment was taken from its real effort value. (3) Finally, in the case of the global estimation model, to estimate the required effort to construct the second increment, we used the value for the real effort that was used for the construction of the first increment. Table 3 shows the difference between estimations from the global estimation model and Incremental CFP (I-CFP).

Table 3. Comparison between Incremental CFP (I-CFP) and Global Estimation for the Second Increment

	Real Effort (m-h)	Estimated Effort Global (m-h)	Estimated Effort I-CFP (m-h)	MRE Global	MRE I-CFP
A	486	564.69	421.53	16.2%	13.3%
B	483	657.02	501.50	36.0%	3.8%

As it can be observed from Table 3, the MRE value for Incremental CFP is less than 13.3%, whereas the value for the global estimation model is 16.2% and even more. This comparison shows that Incremental CFP offers more accurate estimations, regarding effort values. This fact alone proves that the global estimation model might not be suitable for planning purposes [4] in these kinds of scenarios. Based on the results presented here, we can conclude that Incremental CFP provides a better mechanism to estimate effort among increments than a generic global estimation model.

6. CONCLUSIONS AND FUTURE WORKS

This paper presents a technique that is based on Cosmic Function Points to estimate the effort in projects that follows an incremental lifecycle. This technique can be complemented with the Use Case Precedence Diagram in order to prioritize use cases and calculate CFPs that allow project managers to determine the effort required to construct each increment of the project.

The first step in the Incremental CFP technique is to determine the CFPs for each increment. The second step is to define which use cases are going to be constructed in each increment and calculate the effort required. Finally, the third step is to evaluate what occurred during the construction of each increment in order to perform the necessary adjustments to effort calculation. Incremental CFP does not make use of exponential estimation formulas as in other estimation models.

We also show the results obtained by applying the formulas proposed by our technique with the information collected from two projects with undergraduate students. The relative error between the estimated effort and real effort in the second increment was less than 30%. By comparing Incremental CFP with other estimation models, we could observe that Incremental CFP delivers better results than a generic global estimation model.

Additionally, it could be confirmed that since productivity was different among teams, it would not be adequate to estimate the effort required for a project if the information from the development team itself is not taken into consideration. Although the obtained results are promising, we still cannot assert that the technique is applicable for other contexts, such as real world projects. This is due to the fact that, in the kind of projects developed by students, we can control additional factors that might affect effort estimation in a critical way, such as personnel rotation, change of requirements, different sequences of requirements construction, and so on.

Agile methodologies are based on incremental software development models. Therefore, as part of our future work, we will test Incremental CFP with these kinds of methodologies, performing an evaluation with seasoned developers in the field to determine if the technique is applicable to them or not.

7. REFERENCES

- [1] Abrahamsson, P., Moser, R., Pedrycz, W., Sillitti, A., and Succi, G. 2007. Effort Prediction in Iterative Software Development Processes -- Incremental Versus Global Prediction Models. In *Proc. of the 1st Int'l Symp. on ESEM*. ESEM 2007. IEEE Computer Society, 344-353. DOI= <http://dx.doi.org/10.1109/ESEM.2007.16>.
- [2] Boehm, B.W., Madachy, R., and Steece, B. 2000. *Software Cost Estimation with Cocomo II*. Prentice-Hall, Upper Saddle River, NJ.
- [3] Conte, S.D., Dunsmore, H.E., and Shen, V.Y. 1986. *Software Engineering Metrics and Models*. Benjamin-Cummings Publishing Co., Inc., Redwood City, CA.
- [4] Hastings, T.E. and Sajeev, A.S.M. 2001. A Vector-Based Approach to Software Size Measurement and Effort Estimation. *IEEE T Software Eng.* 27, 4 (April. 2001), 337-350. DOI= <http://dx.doi.org/10.1109/32.917523>.
- [5] Heričko, M. and Živković, A. 2008. The Size and Effort Estimates in Iterative Development. *Inform Software Tech.* 50, 7-8 (June. 2008), 772-781. DOI= <http://dx.doi.org/10.1016/j.infsof.2007.08.005>.
- [6] Kang, S., Choi, O., and Baik, J. 2010. Model-Based Dynamic Cost Estimation and Tracking Method for Agile Software Development. In *Proc. of the 9th IEEE/ACIS Int'l Conf. on CIS*. ICIS 2010. IEEE Computer Society, 743-748. DOI= <http://dx.doi.org/10.1109/icis.2010.126>.
- [7] Kendall, K.E. and Kendall, J.E. 2013. *Systems Analysis and Design*. Prentice-Hall, Upper Saddle River, NJ.
- [8] Mohagheghi, P., Anda, B., and Conradi, R. 2005. Effort Estimation of Use Cases for Incremental Large-Scale Software Development. In *Proc. of the 27th Int'l Conf. on SE*. ICSE 2005. IEEE Computer Society, 303-311. DOI= <http://dx.doi.org/10.1109/icse.2005.1553573>.
- [9] Orr, G. and Reeves, T.E. 2000. Function Point Counting: One Program's Experience. *J Syst Software.* 53, 3 (September. 2000), 239-244. DOI= [http://dx.doi.org/10.1016/S0164-1212\(00\)00015-7](http://dx.doi.org/10.1016/S0164-1212(00)00015-7).
- [10] Pow-Sang, J.A. and Jolay-Vasquez, E. 2006. An Approach of a Technique for Effort Estimation of Iterations in Software Projects. In *Proc. of the 13th ASPEC*. APSEC 2006. IEEE Computer Society, 367-376. DOI= <http://dx.doi.org/10.1109/APSEC.2006.20>.
- [11] Pow-Sang, J.A., Nakasone, A., Imbert, R., and Moreno, A.M. 2008. An Approach to Determine Software Requirement Construction Sequences Based on Use Cases. In *Proc. of the ASE*. ASE 2008. IEEE Computer Society, 17-22. DOI= <http://dx.doi.org/10.1109/ase.2008.33>.
- [12] The Common Software Measurement International Consortium. 2014. *The COSMIC Functional Size Measurement Method - version 4.0 Measurement Manual* (The COSMIC Implementation Guide for ISO/IEC 19761: 2011).